Fact-Checking Pipeline for Detecting Hallucination in LLMs

Min Ko
mk9803@g.rit.edu
Rochester Institute of Technology
Rochester, New York, USA

Abstract

Large Language Models (LLMs) have become widely adopted for their ability to generate human-like text and even reason through complex problems. However they are still prone to hallucinations, correct-sounding outputs that are factually-incorrect, making them difficult to trust. Existing solutions grounds the LLMs during generation but do not verify the final responses.

This paper presents a fact-checking pipeline designed to detect hallucinations in LLM outputs and provide transparent evidence. The pipeline first condense the LLM's response into shorter, "summarized" facts that need to be checked. These claims are then used as queries for retrieval from the knowledge corpus to obtain supporting evidence. Finally, a fact-verification model detects if the ground truth supports or refutes the claim to inform the user of possible hallucinations in the LLM's response.

Keywords: LLM, Hallucination, Fact Extraction, Fact Verification, Knowledge Base,

ACM Reference Format:
Min Ko. 2025. Fact-Checking Pipeline for Detecting Hallucination in LLMs. In . ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) have shown great ability to generate coherent, well-written text. Recently, reasoning models like GPT-5 and Gemini 2.5 Pro have even integrated "thinking" into their generation, performing a long chain of thought before creating a final response. These advancements have improved LLMs' ability to "think" through complex prompts to generate a seemingly correct and well-thought-out answers, encouraging more adoption of LLM use in both industry and daily life [14].

Despite these advancements, hallucinations, or when LLMs generate responses that sound correct but are factually-incorrect, remains a persistent and critical challenge [3]. Research by OpenAI have shown that their reasoning models such as gpt-5-thinking and OpenAI o-mini in fact hallucinate more than the previous non-reasoning models [7]. As a user, it is difficult to trust these responses and requires manual fact checking. Existing solutions such as Retrieval-Augmented Generation (RAG) [5] support LLMs by providing information to base the responses on during generation, but fails to verify the final responses as factually-correct. This lack of after-generation verification justifies the need for a pipeline that combines fact extraction, evidence retrieval and fact verification to detect LLM hallucinations with supporting evidence.

The fact-checking pipeline consists four stages that take a LLM response and outputs any hallucinated fact with evidence:

1. **Fact Extraction**: Extract atomic facts from a LLM response
2. **Knowledge Base**: Compile trusted sources to create the central knowledge base
3. **Evidence Retrieval**: Use extracted facts as queries to knowledge base and retrieve supporting evidence
4. **Fact Verification**: Classify extracted facts as *supported*, *refuted*, or *unverifiable* using retrieved evidence.

Fact extraction is implemented using Google's Flan-T5 (Fine-tuned LAnguage Net T5) model [2] that is fine-tuned to this specific task using the Fact Extraction and VERification (FEVER) dataset [12]. The FEVER dataset contains individual facts extracted from Wikipedia which are manually annotated as "Supported", "Refuted" or "NotEnoughInfo" with corresponding evidence. The knowledge base is built as a vector database using trusted sources such as Wikipedia. As such sources may not have information that is recent or obscure, a web searching API updates the knowledge base with new trusted information if that information is not available. Using the extracted facts as queries, supporting evidence is retrieved using two-stage retrieval and a reranker to find the best matched evidence[9]. Finally, fact verification uses a fine-tuned NLI model that determines whether the evidence

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

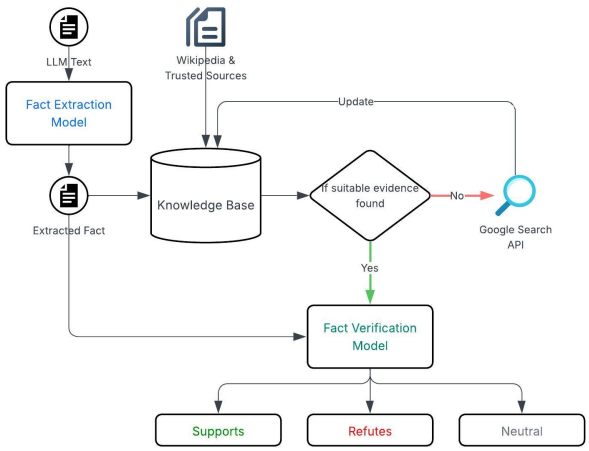

Figure 1. Fact-Checking Pipeline

supports or refutes the corresponding fact and displays the evidence. [PLACEHOLDER FOR PIPELINE EVALUATION]

This paper is organized as follows. Section 2 presents the proposed pipeline. Section 3 reviews the related work in fact verification and hallucination detection. Section 4 presents the evaluation methods and results. Section 5 concludes the paper and discuss key findings and future work.

2 Fact-Checker Pipeline [PLACEHOLDER NAME]

The overview of the proposed fact-checking pipeline is shown in Figure 1.

2.1 Fact Extraction

The first stage of the pipeline decomposes LLM outputs into atomic claims, where each claim representing a single, verifiable fact. To achieve this, we use Google's Flan-T5 transformer model [2], fine-tuned specifically for fact extraction.

The T5 model [8] was developed in 2020 to test the limits of transfer learning in natural language processing (NLP) by introducing a framework that converted all text-related task into a text-to-text format and eliminated the need to train separate, task-specific models. Trained on the Colossal Clean Crawled Corpus (C4) dataset with 745GB of text, the T5 model has a deep and varied understanding of English that is effectively transferred to new, specific tasks through fine-tuning, achieving performance comparable to or exceeding state-of-the-art models at the time across multiple NLP benchmarks. Flan-T5 improves upon this framework by using a more advanced instruction-finetuning process, introducing a wider variety of task instructions and achieving up to 25% increase for certain benchmarks [2].

To finetune this model for fact extraction, we use the FEVER dataset [12], which contains facts that are annotated as "Supported", "Refuted" or "NotEnoughInfo" with the corresponding evidence from Wikipedia. This data is further processed by merging facts that corresponds to the same evidence to ensure that the evidence is decomposed completely into facts. This step enables the model to handle a wide variety of input and output text size, avoiding bias towards extracting a certain number of facts.

To supplement FEVER, a supporting dataset of 5,000 LLM-generated text is added to increase the variety of the data. The LLM outputs are split into atomic claims and manually annotated to ensure the quality, helping the model generalize to the style of LLM-generated content.

The Flan-T5 model is then trained on this combined dataset, which balances high-quality human-annotated facts with diverse, realistic LLM outputs, resulting in a fine-tuned model capable of reliably extracting verifiable claims from complex LLM responses. The results show [PLACEHOLDER FOR COMPONENT EVALUATION].

2.2 Knowledge Base

The second stage of the pipeline builds the initial knowledge base and dynamically adds information that is not present. Text from trusted ground truth sources, such as Wikipedia, are compiled as documents as the initial knowledge base. As recent or niche topics might not be present in such sources, the pipeline uses Google Custom Search JSON API to retrieve reliable information from trusted providers and dynamically updates the knowledge base.

All documents are preprocessed and stored to support hybrid retrieval in stage 3. The documents are split into approximately 500-character chunks without splitting words or sentences in half to maintain the semantic of each chunk. Each chunk is encoded using a sentence transformer [10] model into fixed-length vectors, creating dense representations that captures the semantic, and stored in a vector store, ChromaDB. In parallel, all chunks are also indexed using BM25 [11], a probabilistic keyword-based scoring algorithm that measures the relevance of a chunk to a given query, to create sparse representations that focus on key term frequencies.

By storing both dense and sparse representations, the knowledge base supports retrieval using semantic similarity and textual overlap for a more varied list of evidence used later in the pipeline.

2.3 Evidence Retrieval

The third stage of the pipeline retrieves supporting evidence from the knowledge base to verify the extracted facts from stage 1. The retrieval process uses the two-stage retrieval framework that is widely used in modern RAG systems and popularize by Karpukhin et al. [4–6].

In the first stage, hybrid retrieval is used to get candidate evidence from the knowledge base using the extracted facts as queries. Dense retrieval encodes the query using a

sentence transformer [10] and retrieves its top-k candidates using cosine similarity. To complement this, sparse retriever uses the query to retrieve its top-k candidates with exact keyword matches based on BM25 score.

Dense retrieval provides high recall by identifying contextually relevant evidence even when the query and source text use different wording, while BM25 provides high precision by emphasizing exact lexical overlap. The top candidates from both retrievers are merged to form a combined set for reranking.

In the second stage, the candidate set is passed to a cross-encoder reranker based on a transformer model [6]. This model jointly encodes the query with each candidate evidence, leveraging self-attention to evaluate token-level relationships between the query and evidence. This allows the reranker to distinguish evidence that are truly relevant to the query, providing a better ranking among the candidates to ensure that the most relevant evidence is prioritized for the final stage of the pipeline.

2.4 Fact Verification

The fourth stage of the pipeline verifies each extracted fact with the retrieved evidence using a natural language inference (NLI) model. The NLI task takes a premise and a hypothesis to classify their relationship as *entailment*, *contradiction* or *neutral*. In the context of fact verification, the extracted fact is the hypothesis and the retrieved evidence is the premise.

We use a transformer-based DeBERTa model fine-tuned for NLI tasks on the large-scale, high-quality datasets such as SNLI [1] and MultiNLI [13] datasets. To adapt the model specifically for factual verification, we further fine-tune it on the previously processed FEVER dataset [12].

The verification stage operates as follows. For each fact, each of the candidate evidence from the third stage is encoded together with the fact and passed through the NLI model. The model outputs a confidence score for each of the three NLI labels, indicating the degree to which the evidence supports, refutes, or is unrelated to the fact. The pipeline then aggregates these confidence scores across all evidence to produce a final verification label and confidence score for the fact.

This stage completes the pipeline, allowing it to take an LLM output, decompose it into individual facts, and produce a final evidence-backed verification of each fact.

References

[1] Samuel R Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. 2015. A large annotated corpus for learning natural language inference. *arXiv preprint arXiv:1508.05326* (2015).

[2] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[3] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.

[4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering.. In *EMNLP (1)*. 6769–6781.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[6] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085* (2019).

[7] OpenAI. 2025. *GPT-5 System Card*. https://openai.com/index/gpt-5-system-card/ Accessed: 2025-10-26.

[8] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[9] Jhon Rayo, Raúl de La Rosa, and Mario Garrido. 2025. A Hybrid Approach to Information Retrieval and Answer Generation for Regulatory Texts. *arXiv preprint arXiv:2502.16767* (2025).

[10] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[11] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[12] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and VERification. *arXiv preprint arXiv:1803.05355* (2018).

[13] Adina Williams, Nikita Nangia, and Samuel R Bowman. 2017. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426* (2017).

[14] Fengli Xu, Qianyue Hao, Chenyang Shao, Zefang Zong, Yu Li, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, et al. 2025. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns* 6, 10 (2025).